

Design a Rate Limiter

Atlassian · Microsoft · Oracle · Goldman Sachs · Amazon · Google · PayPal · Adobe – ఈ question LLD round lo class design గా, HLD round lo distributed system గా వస్తుంది.

నాలుగు algorithms ఉన్నాయి, మరియు అవి "ఒకటి మంచిది, మిగతావి చెడ్డవి" కాదు – **నాలుగూ వేర్వేరు trade-offs**. ఇక్కడ మనం ఒక్కొక్కటిగా కట్టి, ఒకే దాడిని నాలుగింటి మీదా ప్రయోగించి, తేడాని సంఖ్యలతో చూస్తాం.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి సంఖ్యా నిజంగా కొలిచినదే – memory numbers, మరియు ఒక 5,000-నమూనాల సరిపోలిక కూడా.

ఈ PROBLEM యొక్క ప్రత్యేకత

మిగతా LLD problems lo "సరైన జవాబు" ఒకటి ఉంటుంది. ఇక్కడ లేదు. Fixed window, sliding log, sliding counter, token bucket – నాలుగు నిజమైన systems lo వాడబడుతున్నాయి. ఒక్కొక్కటి ఒక్కో దాన్ని త్యాగం చేస్తుంది: ఖచ్చితత్వం, memory, లేదా burst అనుమతి.

అందుకే interviewer చూసేది మీరు ఏ algorithm రాశారో కాదు – మీరు ఏది ఎందుకు ఎంచుకున్నారో. ఈ doc అదే నేర్పుతుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Rate limiter ఎందుకు, మరియు నిజమైన ప్రశ్న ఏమిటి
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి version, మరియు మొదటి విరుపు

3. Step – Fixed window counter
4. మొదటి విరుపు – సరిహద్దు వద్ద రెట్టింపు దాడి

Part 3 – రెండో విరుపు: ఖచ్చితమైనది, కానీ ఖరీదైనది

5. Step – Sliding window log
6. రెండో విరుపు – memory ని కొలిచి చూడటం

Part 4 – అంచనా: ఖచ్చితత్వాన్ని కొంచెం వదులుకోవడం

7. Step – Sliding window counter
8. ఇది ఎంత తప్పు? – 5,000 నమూనాల మీద కొలత

Part 5 – Token bucket: burst ని ఉద్దేశపూర్వకంగా అనుమతించడం

9. Step – Token bucket
10. నాలుగు algorithms · ఒకే దాడి · ఒకే పట్టిక

Part 6 – పూర్తి system

11. Step – నాలుగింటినీ ఒకే ఒప్పందం కిందికి
12. Cost, keys, మరియు తిరస్కరణ ఎలా కనిపించాలి

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 7 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటితో చెప్పాల్సిన English script

16. Follow-ups – distributed rate limiting

17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Rate limiter ఎందుకు, మరియు నిజమైన ప్రశ్న ఏమిటి

Rate limiter = "ఒక client ఒక నిర్ణీత కాలంలో ఇంతకంటే ఎక్కువ requests పంపకూడదు" అని అమలు చేసే ఒక కాపలా.

ఇది ఎందుకు కావాలి? మూడు కారణాలు, మూడూ వేరే స్వభావం ఉన్నవి:

- **దుర్వినియోగం ఆపడానికి** – ఒకరు script రాసి సెకనుకి 10,000 requests పంపితే.
- **న్యాయం కోసం** – ఒక భారీ customer మొత్తం సామర్థ్యాన్ని తినేసి మిగతా అందరినీ ఆకలితో ఉంచకుండా.
- **ఖర్చు అదుపు కోసం** – ప్రతి request ఒక LLM call లేదా ఒక payment gateway hit అయితే, hard limit లేకపోతే bill పేలిపోతుంది.

ఇది సులభం అనిపిస్తుంది: "ఒక counter పెట్టి, హద్దు దాటితే ఆపేయడం." అదే మొదటి version (§3), మరియు అది తప్పు.

అసలు ప్రశ్న ఇది: "ఒక నిమిషంలో 5 requests" – ఆ "ఒక నిమిషం" ఎక్కడ మొదలవుతుంది?

ఈ ఒక్క ప్రశ్నకి నాలుగు వేర్వేరు జవాబులు ఉన్నాయి, మరియు అవే నాలుగు algorithms.

"నిమిషానికి 5" - కానీ నిమిషం అంటే ఏమిటి?

Fixed window "గడియారం ప్రకారం 1:00-1:59" Memory: ఒక్క counter సరళం, వేగం X సరిహద్దు వద్ద రెట్టంపు అనుమతిస్తుంది (54)	Sliding log "ఇప్పటి నుంచి వెనక్కి 60 సెకన్లు" Memory: ప్రతి request కీ ఒక timestamp ఖచ్చితంగా సరైనది X memory పీలుతుంది (56)
Sliding counter "ఈ window + గతదాని బరువు కలిపిన భాగం" Memory: రెండు సంఖ్యలు దాదాపు సరైనది ~ ఒక అంచనా - ఎంత తప్పు 58 lo కొలిచాం	Token bucket "నిరంతరం నిండుతున్న ఒక బకెట్" Memory: రెండు సంఖ్యలు Burst ని ఉద్దేశపూర్వకంగా అనుమతిస్తుంది → నిజ systems lo అత్యధికంగా వాడేది

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
దేని ఆధారంగా limit – user, IP, API key?	ఇదే key ని నిర్ణయిస్తుంది. బహుళ స్థాయిలు కావాలంటే బహుళ limiters
Burst అనుమతించాలా, లేక నిరంతరం సమానంగా ఉండాలా?	Burst కావాలంటే token bucket ; లేకపోతే sliding
ఖచ్చితత్వం ఎంత ముఖ్యం?	"సుమారుగా సరిపోతే చాలు" అంటే counter; billing అయితే log
ఎంతమంది users?	ఇదే memory ని నిర్ణయిస్తుంది (56)
ఒకే server నా, పలు servers నా?	ఇది మీరే లేవనెత్తాలి – పలు servers అంటే పూర్తిగా వేరే problem (516)
హద్దు దాటితే ఏం చేయాలి – ఆపాలా, ఆలస్యం చేయాలా, queue lo పెట్టాలా?	తిరస్కరణ (429) vs throttling – వేరే ప్రవర్తనలు
అన్ని endpoints కీ ఒకే limit నా?	కాకపోతే cost అనే భావన కావాలి (512)

"ఒకే SERVER నా, పలు SERVERS నా?" – ఈ ప్రశ్న ఎందుకు అంత ముఖ్యం

ఒకే server అయితే ఇది ఒక **data structure** problem – ఈ doc lo 90% అదే.

పలు servers అయితే ఇది ఒక **distributed state** problem: counter ఎక్కడ ఉంటుంది? Redis నా? అప్పుడు ప్రతి request కీ ఒక network hop. Redis పడిపోతే? Race conditions ఎలా ఆపాలి?

ఈ ప్రశ్న అడగకపోతే మీరు in-memory version రాస్తారు, ఆపై interviewer "ఇప్పుడు 50 servers" అంటాడు, మరియు మీ design మొత్తం తిరిగి ఆలోచించాలి. **అడిగితే** – మీరు scope ని మీరే నిర్ణయించి, 516 ని ఒక planned extension గా ఉంచుకోగలరు.

Part 2 – మొదటి version, మరియు మొదటి విరుపు

3. Step – Fixed window counter

అతి సహజమైన ఆలోచన: గడియారాన్ని నిమిషాలుగా కోసి, ప్రతి నిమిషానికి ఒక counter. నిమిషం మారితే counter సున్నా.

```
class FixedWindow {
  constructor(limit, windowMs, clock) {
    Object.assign(this, { limit, windowMs, clock });
    this.hits = new Map();
  }

  allow(user) {
    const w = Math.floor(this.clock() / this.windowMs); // ఏ నిమిషం lo ఉన్నాం
    const rec = this.hits.get(user);

    if (!rec || rec.window !== w) { // కొత్త నిమిషం → మొదలు
      this.hits.set(user, { window: w, count: 1 });
      return true;
    }
    if (rec.count < this.limit) { rec.count++; return true; }
    return false;
  }
}
```

`Math.floor(now / windowMs)` అనేది ఒక చక్కని ఉపాయం – ఇది ఏ timestamp నైనా ఒక **window సంఖ్య** గా మారుస్తుంది. `windowMs` 60,000 అయితే, 1:30 మరియు 1:45 రెండూ ఒకే window సంఖ్య ఇస్తాయి.

`clock` ని **constructor lo తీసుకోవడం** గమనించండి – `Date.now()` ని లోపల పిలవట్లేదు. ఇది Deep Dive 02 §15 lo చూసిన అదే సూత్రం. Rate limiter lo **ఇది ఇంకా ముఖ్యం**, ఎందుకంటే ఇక్కడ ప్రతి test కీ "ఒక నిమిషం గడిచింది" అని నటించాలి. నిజమైన గడియారంతో ఆ test ఒక నిమిషం ఆగాలి. నడిపి చూద్దాం – నిమిషానికి 5 హద్దుతో, రెండు నిమిషాల్లో 7 చొప్పున:

మొదటి నిమిషం: 7 requests → 5 అనుమతించాం, 2 ఆపాం
రెండో నిమిషం : 7 requests → 5 అనుమతించాం, 2 ఆపాం

సరైనదే అనిపిస్తోంది. ఒక్క నిమిషంలో సరిగ్గా 5. కాబట్టి ఇక్కడ ఆగిపోవచ్చా?

4. మొదటి విరుపు – సరిహద్దు వద్ద రెట్టింపు దాడి

ఇప్పుడు ఒక తెలివైన దాడి చేద్దాం. మనం requests ని నిమిషం సరిహద్దు చుట్టూ గుమిగూడ్చుదాం – 1:59.900 వద్ద ఐదు, 2:00.100 వద్ద ఐదు.

```
let now = 119_900; // 1:59.900
const rl = new FixedWindow(5, 60_000, () => now);
let allowed = 0;

now = 119_900; for (let i = 0; i < 5; i++) if (rl.allow('mallory')) allowed++;
now = 120_100; for (let i = 0; i < 5; i++) if (rl.allow('mallory')) allowed++;
```

హద్దు: నిమిషానికి 5
1:59.900 → 5 requests · 2:00.100 → 5 requests
మొత్తం అనుమతించినవి: 10 (200 milliseconds lo!)

మొదటి విరుపు – హద్దుకి రెట్టింపు

"నిమిషానికి 5" అని చెప్పాం. కానీ ఒక client **200 millisecondsలో 10** పంపగలిగాడు – హద్దుకి రెట్టింపు.

Code lo bug లేదు. ప్రతి window lo సరిగ్గా 5 అనుమతించాం. తప్పు ఏమిటంటే – మన "నిమిషం" గడియారానికి అతుక్కుని ఉంది, client కి కాదు. Client తన నిమిషాన్ని 1:59.900 నుంచి 2:00.900 గా లెక్కిస్తాడు, మరియు ఆ నిమిషంలో అతను 10 పంపాడు.

ఇది ఒక సైద్ధాంతిక సమస్య కాదు – ఇది **fixed window rate limiters మీద అత్యంత సాధారణ దాడి**, మరియు దీనికి ఒక పేరు కూడా ఉంది: **boundary burst**.

సరైన ప్రశ్న

Fixed window అడుగుతేంది: "ఈ గడియారపు నిమిషంలో ఎన్ని పంపావు?"

అడుగాల్సినది: "ఇప్పటి నుంచి వెనక్కి 60 సెకన్లలో ఎన్ని పంపావు?"

ఈ ఒక్క మార్పు – "గడియారపు నిమిషం" నుంచి "కదిలే 60 సెకన్లు" కి – మొత్తం data structure ని మారుస్తుంది. ఎందుకంటే ఇప్పుడు మనకి **ఎప్పుడు** పంపారో గుర్తుండాలి, కేవలం ఎన్ని అని కాదు.

Part 3 – రెండో విరుపు: ఖచ్చితమైనది, కానీ ఖరీదైనది

5. Step – Sliding window log

జవాబు స్పష్టం: ప్రతి request యొక్క timestamp ని దాచుకుందాం. అడిగినప్పుడు, 60 సెకన్ల కంటే పాతవాటిని వదిలేసి, మిగిలినవి లెక్కిద్దాం.

```
class SlidingLog {
  constructor(limit, windowMs, clock) {
    Object.assign(this, { limit, windowMs, clock });
    this.log = new Map(); // user → timestamps[]
  }

  allow(user) {
    const now = this.clock(), cutoff = now - this.windowMs;
    const times = this.log.get(user) ?? [];

    while (times.length && times[0] <= cutoff) times.shift(); // పాతవి వదిలేయడం

    if (times.length >= this.limit) { this.log.set(user, times); return false; }
    times.push(now); this.log.set(user, times);
    return true;
  }
}
```

ఇది ఏం చేస్తుంది: `times` ఎప్పుడూ క్రమంలో ఉంటుంది (కొత్తవి చివర చేరతాయి), కాబట్టి పాతవి ఎప్పుడూ మొదట్లోనే ఉంటాయి. అందుకే మొదటి నుంచి `shift` చేస్తూ పోవడం సరిపోతుంది.

అదే సరిహద్దు దాడిని ప్రయోగిద్దాం:

```
అదే దాడి → అనుమతించినవి: 5 (fixed window lo 10 వచ్చింది)
60 సెకన్ల తర్వాత మళ్ళీ: true
```

5. ఖచ్చితంగా సరైనది. మొదటి విరుపు పూర్తిగా సరిచేయబడింది. మరియు 60 సెకన్ల తర్వాత మళ్ళీ అనుమతి వచ్చింది – అంటే window నిజంగా కదులుతోంది.

ఇది ఖచ్చితమైన జవాబు. ఇంతకంటే మంచిది ఏమీ లేదు. కాబట్టి ఇదే వాడితే సరిపోతుంది కదా?

6. రెండో విరుపు – memory ని కొలిచి చూడటం

"ఇది ఎక్కువ memory తింటుంది" అని చెప్పడం సులభం. కొలిచి చూపిద్దాం.

```
for (const [users, perUser] of [[1000,100],[10000,100],[50000,100]]) {
  global.gc();
  const before = process.memoryUsage().heapUsed;
  let now = 0;
  const rl = new SlidingLog(perUser, 60_000, () => now);
  for (let u = 0; u < users; u++)
    for (let i = 0; i < perUser; i++) { now = i; rl.allow('u' + u); }
  const used = (process.memoryUsage().heapUsed - before) / 1048576;
  console.log(`${String(users).padStart(6)} users × ${perUser} req →
    ${used.toFixed(1)} MB`);
}
```

1000 users × 100 req → 2.2 MB
10000 users × 100 req → 13.3 MB
50000 users × 100 req → 67.1 MB

ఈ సంఖ్యల గురించి: ఇవి ఒక నిజమైన run నుంచి (`node --expose-gc`). GC ఎప్పుడు నడిచిందో బట్టి ప్రతి run lo ± 0.2 MB తేడా వస్తుంది. **ఖచ్చితమైన సంఖ్య కాదు, పెరుగుదల నమూనా ముఖ్యం** – మరియు అది ప్రతి run lo ఒకటే.

రెండో విరుపు – ఇది రేఖీయంగా పెరుగుతోంది

Users 50 రెట్లు పెరిగితే memory 30 రెట్లు పెరిగింది. ఇది **$O(\text{users} \times \text{requests})$** .

ఇప్పుడు నిజమైన సంఖ్యలు పెడదాం: **10 లక్షల users**, ఒక్కొక్కరికి నిమిషానికి 100 requests. పై నిష్పత్తి ప్రకారం అది **~1.3 GB** – కేవలం rate limiting కోసం. మరియు ఇది **ప్రతి server మీదా**.

మరియు memory ఒక్కటే సమస్య కాదు: ప్రతి `allow` కీ `shift()` చేస్తున్నాం, అది array మొదటి నుంచి తీయడం – పెద్ద arrays కి ఇది కూడా ఖరీదు.

ఇక్కడ ఒక ముఖ్యమైన గుర్తింపు

మనకి నిజంగా **ప్రతి timestamp** కావాలా? లేదు – మనకి కావలసినది ఒక్కటే: **"గత 60 సెకన్లలో ఎన్ని?"** ఆ సంఖ్య చెప్పడానికి అన్ని timestamps అవసరం లేదు.

అంటే మనం **ఖచ్చితత్వాన్ని కొంచెం వదులుకుని** memory ని భారీగా తగ్గించగలమా? – ఇదే తర్వాతి Part.

Part 4 – అంచనా: ఖచ్చితత్వాన్ని కొంచెం వదులుకోవడం

7. Step – Sliding window counter

అలోచన: **రెండు counters మాత్రమే ఉంచుదాం** – ఈ window కి ఒకటి, గత window కి ఒకటి. ఆపై "గత 60 సెకన్లు" అనేదాన్ని ఒక **బరువు కలిపిన అంచనా** గా లెక్కిద్దాం.

ఇప్పుడు 2:00:15 అనుకుందాం (ఈ window lo 25% గడిచింది). అప్పుడు "గత 60 సెకన్లు" అంటే:

- ఈ window lo ఇప్పటిదాకా వచ్చినవన్నీ, **పూర్తిగా**
- గత window నుంచి **75%** (ఎందుకంటే ఆ window lo చివరి 75% మన 60-సెకన్ల పరిధిలో ఉంది)

```
class SlidingCounter {
  constructor(limit, windowMs, clock) {
    Object.assign(this, { limit, windowMs, clock });
    this.rec = new Map();
  }

  allow(user) {
    const now = this.clock(), w = Math.floor(now / this.windowMs);
    const elapsed = (now % this.windowMs) / this.windowMs; // 0..1 - ఎంత గడిచింది

    let r = this.rec.get(user);
    if (!r || r.window < w - 1) r = { window: w, curr: 0, prev: 0 }; // రెండు
    పాతవి
    else if (r.window === w - 1) r = { window: w, curr: 0, prev: r.curr }; // ఒకటి
    జరిగింది

    const estimate = r.curr + r.prev * (1 - elapsed); // ← మొత్తం ఉపాయం ఇదే

    if (estimate + 1 > this.limit) { this.rec.set(user, r); return false; } // +1 =
    ఈ request
    r.curr++; this.rec.set(user, r);
    return true;
  }
}
```

Memory చూద్దాం – sliding log తో అదే పరీక్ష:

50000 users × 100 req → 14.5 MB (log lo 67.1 MB)

67.1 MB → 14.5 MB. మరియు ముఖ్యమైనది: ఇది ఇక **requests సంఖ్యతో పెరగదు** – ఒక్కో user కి ఎప్పుడూ రెండే సంఖ్యలు. 10 లక్షల users కి కూడా ఇది ఒక స్థిరమైన, చిన్న సంఖ్య.

8. ఇది ఎంత తప్పు? – 5,000 నమూనాల మీద కొలత

ఇది ఒక అంచనా. అంటే ఎక్కడో తప్పు ఉంటుంది. ఎంత తప్పు, మరియు ఏ దిక్కులో?

చాలా చోట్ల ఇలా రాస్తారు: "sliding counter కొంచెం ఎక్కువ అనుమతించొచ్చు." నేను దాన్ని నమ్మకుండా

కొలిచాను – 5,000 యాదృచ్ఛిక traffic నమూనాలని రెండు algorithms మీదా నడిపి, తేడా చూశాను:

5000 నమూనాలు: సగటు తేడా -1.20 · గరిష్ట ఎక్కువ 1 · గరిష్ట తక్కువ -5

కొలత ఒక అపోహని సరిచేసింది

`estimate + cost > limit` అనే రూపంలో, sliding counter **ఎక్కువ కాదు – తక్కువ** అనుమతిస్తుంది. 5,000 నమూనాల్లో సగటున **1.20 requests తక్కువ**, గరిష్టంగా 5 తక్కువ. ఎక్కువ అనుమతించినది గరిష్టంగా **1** మాత్రమే.

ఎందుకు? అంచనా "గత window lo requests సమానంగా పంచబడ్డాయి" అని ఊహిస్తుంది. నిజ traffic lo అవి గుంపులుగా ఉంటాయి. ఆ గుంపు window మొదట్లో ఉంటే – అవి నిజంగా మన 60-సెకన్ల పరిధి బయట ఉన్నాయి, కానీ అంచనా వాటిలో కొంత భాగాన్ని లెక్కిస్తుంది. అంటే అది **ఎక్కువగా అంచనా వేసి, ముందుగానే ఆపేస్తుంది**.

ఇది మంచి వార్త: ఒక rate limiter కొంచెం కఠినంగా ఉండటం, కొంచెం వదులుగా ఉండటం కంటే సురక్షితం. కానీ మీరే కొలవకుండా ఈ దిశని ఊహించకండి – అది `>=` నా `>` నా అనే ఒక్క అక్షరం మీద ఆధారపడుతుంది.

Interview lo ఇది బంగారం. "The sliding window counter is an approximation. I measured it against the exact log over five thousand random traffic patterns – with this comparison it came out about 1.2 requests stricter on average, never more than 1 request looser. For a rate limiter, erring strict is the safe direction." – ఇలాంటి ఒక వాక్యం మిమ్మల్ని "చదివి గుర్తుపెట్టుకున్నవాడు" నుంచి "పరీక్షించి చూసినవాడు" గా మారుస్తుంది.

Part 5 – Token bucket: burst ని ఉద్దేశపూర్వకంగా అనుమతించడం

9. Step – Token bucket

ఇప్పటిదాకా మనం burst ని ఒక **దాడి** గా చూశాం. కానీ ఒక్క క్షణం ఆలోచించండి – burst ఎప్పుడూ చెడ్డదేనా?

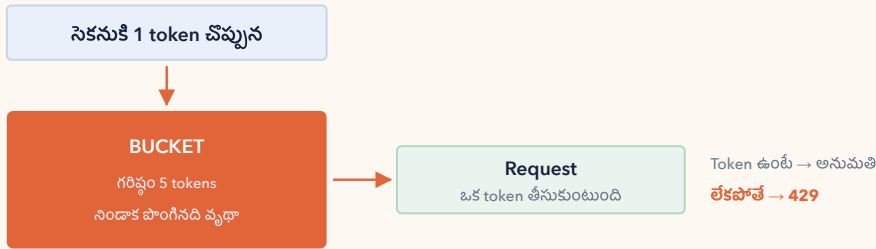
ఒక mobile app ఆలోచించండి. User దాన్ని తెరిచినప్పుడు అది ఒకసారి 8 API calls చేస్తుంది – profile, feed, notifications, settings. ఆపై అతను చదువుతూ ఐదు నిమిషాలు ఏమీ చేయడు.

Sliding window తో "నిమిషానికి 10" అని పెడితే, ఆ 8 calls **బాగానే** పోతాయి. కానీ "నిమిషానికి 5" అయితే app తెరవగానే విరిగిపోతుంది – అయినా ఆ user సగటున చాలా తక్కువ traffic పంపుతున్నాడు.

మనకి కావలసినది: **సగటుని అదుపులో ఉంచుతూ, అడపాదడపా burst ని అనుమతించడం. అదే token bucket.**

TOKEN BUCKET • ఒక బకెట్, ఒక కుళాయి

ఒక్క request ఒక token తింటుంది • tokens నిరంతరం తిరిగి నిండుతాయి



రెండు సంఖ్యలే మొత్తం design ని నిర్ణయిస్తాయి

capacity = ఎంత burst అనుమతించాలి • **refill rate** = దీర్ఘకాలిక సగటు ఎంత

```
class TokenBucket {
  constructor(capacity, refillPerSec, clock) {
    Object.assign(this, { capacity, refillPerSec, clock });
    this.b = new Map();
  }

  allow(user, cost = 1) {
    const now = this.clock();
    let s = this.b.get(user) ?? { tokens: this.capacity, last: now }; // కొత్తవాడు → నిండిన బకెట్

    const refill = ((now - s.last) / 1000) * this.refillPerSec;
    s.tokens = Math.min(this.capacity, s.tokens + refill); // హద్దు దాటి నిండదు
    s.last = now;

    if (s.tokens >= cost) { s.tokens -= cost; this.b.set(user, s); return true; }
```

```

    this.b.set(user, s);
    return false;
  }
}

```

ఇక్కడ ఒక చక్కని ఉపాయం ఉంది

Tokens ని **timer తో నింపట్లేదు**. బదులుగా – అడిగినప్పుడు "చివరిసారి నుంచి ఎంత సమయం గడిచింది?" అని లెక్కించి, అప్పుడు నింపుతున్నాం. దీన్ని **lazy refill** అంటారు.

ఇది ఎందుకు ముఖ్యం: 10 లక్షల users ఉంటే, timer-based refill అంటే **10 లక్షల timers**. Lazy refill అంటే **సున్నా timers** – ఖర్చు కేవలం ఒక తీసివేత, అది కూడా ఆ user వచ్చినప్పుడు మాత్రమే.

Interview lo ఇది చెప్పండి: "I refill lazily – I compute how many tokens should have accrued since the last request instead of running a timer per user. With a million users, timers would be the whole problem."

నడిపి చూద్దాం – capacity 5, సెకనుకి 1 token:

```

t=0 (నిండిన bucket)      7 req → 5 అనుమతి • మిగిలిన tokens: 0
t=2s (2 tokens తిరిగొచ్చాయి) 3 req → 2 అనుమతి • మిగిలిన tokens: 0
t=10s (పూర్తిగా నిండింది) 6 req → 5 అనుమతి • మిగిలిన tokens: 0

```

మూడు పంక్తులూ ఆలోచనని పూర్తిగా వివరిస్తాయి:

- **t=0** – బకెట్ నిండి ఉంది, కాబట్టి **5 burst** అనుమతి. ఇది ఒక లక్షణం, ఒక bug కాదు.
- **t=2s** – రెండు సెకన్లలో సరిగ్గా 2 tokens తిరిగొచ్చాయి, కాబట్టి 2 అనుమతి.
- **t=10s** – 10 tokens రావాలి, కానీ capacity 5. **Math.min** వల్ల **5 వద్దే ఆగిపోయింది** – లేకపోతే ఒక user ఒక రోజు ఆగి, 86,400 tokens పోగేసుకుని ఒకేసారి పంపేవాడు.

10. నాలుగు algorithms • ఒకే దాడి • ఒకే పట్టిక

ఇప్పుడు నాలుగింటినీ **సరిగ్గా అదే** సరిహద్దు దాడికి గురిచేద్దాం – నిమిషానికి 5 హద్దు, 1:59.900 వద్ద ఐదు, 2:00.100 వద్ద ఐదు:

```

హద్దు: నిమిషానికి 5 • దాడి: సరిహద్దు వద్ద 5 + 5
FixedWindow    → 10
SlidingLog      → 5
SlidingCounter → 5
TokenBucket     → 5

```

Algorithm	దాడిలో	Memory (50k users)	ఖచ్చితత్వం	Burst	ఎప్పుడు వాడాలి
Fixed window	10 x	అతి తక్కువ	కచ్చితంగా తప్పు అంచుల్లో	ప్రమాదకరం	దాదాపు ఎప్పుడూ వద్దు
Sliding log	5 ✓	67.1 MB	ఖచ్చితం	లేదు	Billing, తక్కువ users
Sliding counter	5 ✓	14.5 MB	~1.2 కఠినం	లేదు	సాధారణ API limits
Token bucket	5 ✓	14.5 MB తరహా	ఖచ్చితం	ఉద్దేశపూర్వకం	User-facing APIs

ఈ పట్టికని INTERVIEW LO గీయండి

"ఏ algorithm వాడతారు?" అని అడిగితే – ఒక పేరు చెప్పడం ఒక జవాబు. **ఈ పట్టిక** ఒక వేరే స్థాయి జవాబు, ఎందుకంటే అది "ఏది ఎందుకు" అని చూపిస్తుంది.

మరియు సిఫార్సు స్పష్టం: **user-facing API కి token bucket**. కారణం – నిజమైన users burst గా వస్తారు (app తెరవడం, page refresh), మరియు token bucket ఆ burst ని సగటుని దెబ్బతీయకుండా అనుమతిస్తుంది. అందుకే Stripe, GitHub, AWS – అన్నీ ఏదో ఒక రూపంలో దీన్నే వాడతాయి.

Part 6 – పూర్తి system

11. Step – నాలుగింటినీ ఒకే ఒప్పందం కిందికి

నాలుగు algorithms ని పక్కపక్కన పెట్టి చూడండి. అన్నిటికీ ఒకే ప్రశ్న, ఒకే జవాబు:

"ఈ key, ఈ సమయంలో, ఈ ఖర్చుతో – అనుమతించాలా?" → `true` / `false`

అంటే ఒప్పందం ఒక్క method:

```
class RateLimitStrategy {
  tryAcquire(key, now, cost) { throw new Error('subclass implement చేయాలి'); }
}
```

`now` ని **parameter** గా ఇస్తున్నాం, strategy లోపల `Date.now()` పిలవట్లేదు. అంటే గడియారం ఒకే చోట – `RateLimiter` lo. నాలుగు strategies నీ ఒకే నకిలీ గడియారంతో test చేయొచ్చు.

12. Cost, keys, మరియు తిరస్కరణ ఎలా కనిపించాలి

Cost – అన్ని requests ఒకేలా కావు

`/search` ఒక చౌకైన call. `/export` ఒక భారీ report తయారుచేస్తుంది. రెండింటినీ "ఒక request" గా లెక్కించడం తప్పు.

పరిష్కారం: ప్రతి request కి ఒక **cost**. Token bucket lo ఇది సహజంగా సరిపోతుంది – ఖరీదైన call ఎక్కువ tokens తింటుంది.

```
/search (cost 1) → true
/export (cost 3) → true
/export (cost 3) → true
/export (cost 3) → true
/export (cost 3) → false
stats: { allowed: 4, rejected: 1 }
```

Capacity 10; మొత్తం ఖర్చు $1+3+3+3 = 10$, కాబట్టి ఐదో call కి tokens లేవు. **ఒక limiter, రెండు వేర్వేరు బరువుల endpoints.**

Key – దేని ఆధారంగా limit

`keyOf` ని కూడా బయట పెట్టాం, ఎందుకంటే ఇది సందర్భాన్ని బట్టి మారుతుంది:

ఎమి limit చేస్తున్నాం	keyOf
Login attempts	<code>r => r.ip</code>
API quota	<code>r => r.apiKey</code>
User actions	<code>r => r.userId</code>
Endpoint-వారీ	<code>r => `\${r.userId}:\${r.path}`</code>

నిజ systems లో **బహుళ limiters** ఒకేసారి నడుస్తాయి – IP కి ఒకటి, API key కి ఒకటి, global ఒకటి. Request అన్నిటినీ దాటాలి. మన design లో అది కేవలం మూడు `RateLimiter` objects.

తిరస్కరణ – `false` సరిపోదు

`allow()` కేవలం `true` / `false` ఇస్తే, caller కి ఎంతసేపు ఆగాలో తెలియదు. HTTP లో సరైన జవాబు **429 Too Many Requests** + ఒక `Retry-After` :

```
{ allowed: false, status: 429, retryAfterMs: 1000 }
```

ఇది ఒక చిన్న వివరం, కానీ interviewer గమనిస్తాడు. `Retry-After` లేకపోతే clients వెంటనే మళ్ళీ ప్రయత్నిస్తాయి – అంటే మీ server ఇప్పుడు తిరస్కరణలతో మునిగిపోతుంది. Rate limiter ఉండి కూడా load తగ్గదు. తిరస్కరణ ఒక సమాచారం ఇవ్వాలి, కేవలం ఒక తలుపు మూయడం కాదు.

13. మొత్తం code ఒకే చోట • నడిపి చూద్దాం

```
class RateLimitStrategy {
  tryAcquire(key, now, cost) { throw new Error('subclass implement చేయాలి'); }
}

class FixedWindow extends RateLimitStrategy {
  #hits = new Map();
  constructor(limit, windowMs) { super(); Object.assign(this, { limit, windowMs }); }
  tryAcquire(key, now, cost = 1) {
    const w = Math.floor(now / this.windowMs);
    let r = this.#hits.get(key);
    if (!r || r.window !== w) r = { window: w, count: 0 };
    if (r.count + cost > this.limit) { this.#hits.set(key, r); return false; }
    r.count += cost; this.#hits.set(key, r); return true;
  }
}

class SlidingLog extends RateLimitStrategy {
  #log = new Map();
  constructor(limit, windowMs) { super(); Object.assign(this, { limit, windowMs }); }
  tryAcquire(key, now, cost = 1) {
    const cutoff = now - this.windowMs;
    const times = this.#log.get(key) ?? [];
```

```

    while (times.length && times[0] <= cutoff) times.shift();
    if (times.length + cost > this.limit) { this.#log.set(key, times); return false;
  }
  for (let i = 0; i < cost; i++) times.push(now);
  this.#log.set(key, times); return true;
}
}

class SlidingCounter extends RateLimitStrategy {
  #rec = new Map();
  constructor(limit, windowMs) { super(); Object.assign(this, { limit, windowMs }); }
  tryAcquire(key, now, cost = 1) {
    const w = Math.floor(now / this.windowMs);
    const elapsed = (now % this.windowMs) / this.windowMs;
    let r = this.#rec.get(key);
    if (!r || r.window < w - 1) r = { window: w, curr: 0, prev: 0 };
    else if (r.window === w - 1) r = { window: w, curr: 0, prev: r.curr };
    const estimate = r.curr + r.prev * (1 - elapsed);
    if (estimate + cost > this.limit) { this.#rec.set(key, r); return false; }
    r.curr += cost; this.#rec.set(key, r); return true;
  }
}

class TokenBucket extends RateLimitStrategy {
  #buckets = new Map();
  constructor(capacity, refillPerSec) { super(); Object.assign(this, { capacity, refillPerSec }); }
  tryAcquire(key, now, cost = 1) {
    let s = this.#buckets.get(key) ?? { tokens: this.capacity, last: now };
    s.tokens = Math.min(this.capacity, s.tokens + ((now - s.last) / 1000) * this.refillPerSec);
    s.last = now;
    if (s.tokens < cost) { this.#buckets.set(key, s); return false; }
    s.tokens -= cost; this.#buckets.set(key, s); return true;
  }
}

class RateLimiter {
  #allowed = 0; #rejected = 0;
  constructor(strategy, { clock = () => Date.now(), keyOf = (r) => r.userId } = {}) {
    Object.assign(this, { strategy, clock, keyOf });
  }
  check(request, cost = 1) {
    const ok = this.strategy.tryAcquire(this.keyOf(request), this.clock(), cost);
    ok ? this.#allowed++ : this.#rejected++;
    return ok ? { allowed: true }
      : { allowed: false, status: 429, retryAfterMs: this.strategy.windowMs ?? 1000 };
  }
  get stats() { return { allowed: this.#allowed, rejected: this.#rejected }; }
}

```

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>limit</code> , <code>windowMs</code> , <code>clock</code>	మౌలిక అస్థిపంజరం

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§4 (విరుపు)	Timestamps గుర్తుపెట్టుకోవడం	సరిహద్దు వద్ద రెట్టింపు
§6 (విరుపు)	రెండు counters మాత్రమే	67 MB memory
§9	Lazy refill, <code>Math.min</code> cap	Burst ని అనుమతిస్తూ సగటుని అదుపులో ఉంచడం
§11	<code>RateLimitStrategy</code> ఒప్పందం	నాలుగింటి ఆకారం ఒకటే
§12	<code>cost</code> , <code>keyOf</code> , 429 + <code>retryAfterMs</code>	నిజమైన API కి కావలసినవి

Part 7 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

ఇక్కడ ఉపాయం: అన్నీ రాయొద్దు – రెండు రాసి, మిగతావి మాటల్లో చెప్పండి

6 నిమి

Clarify – "ఒకే server నా పలు servers నా?" తప్పనిసరి

5 నిమి

Fixed window రాసి, దాని విరుపుని చూపించడం

12 నిమి – Sliding log

ఖచ్చితం, కానీ memory · ఇక్కడే trade-off కథ

14 నిమి – Token bucket

ఇదే మీ సిఫార్సు – పూర్తిగా రాయండి

8 నిమి

Strategy + distributed గురించి మాటలు

Sliding counter ని రాయొద్దు – దాన్ని ఒక్క వాక్యంలో వివరించండి: "There's a middle option – keep two counters and weight the previous window. Constant memory, slightly approximate." అది చాలు.

ఏమి తప్పక చెప్పాలి

1. **Fixed window యొక్క boundary burst** (§4) – మీరే దాన్ని చూపించాలి, interviewer అడిగేవరకు కాదు.
2. **Sliding log యొక్క memory** (§6) – ఒక నిజమైన సంఖ్యతో ("10 లక్షల users కి ~1.3 GB").
3. **Token bucket ఎందుకు మీ సిఫార్సు** (§10) – నిజమైన users burst గా వస్తారు.
4. **Lazy refill** (§9) – timers లేకపోవడం ఒక design నిర్ణయం.
5. **429 + Retry-After** (§12) – తిరస్కరణ కూడా ఒక API.

15. నీటితో చెప్పాల్సిన English script

"First – is this one server or many? That changes the problem completely, so I'll build the single-node version and then talk about what breaks when it's distributed. And what are we keying on – user, IP, API key?

The naive answer is a counter per minute. That's wrong, and I can show you why: with a limit of five per minute, I can send five at 1:59.9 and five at 2:00.1 and get ten through in two hundred milliseconds. The window is pinned to the clock, not to the caller.

The exact fix is a sliding log – store every timestamp, drop anything older than the window, count what's left. That's correct, but I measured the memory: fifty thousand users at a hundred requests each is about 67 MB, and it grows with request volume, not just user count. At a million users that's over a gigabyte, per server.

So there's a middle option – keep two counters, current and previous window, and weight the previous one by how far into the current window you are. Constant memory per user. It's an approximation; I actually measured it against the exact log over five thousand random traffic patterns and it came out about 1.2 requests stricter on average, never more than one looser. Erring strict is the right direction for a limiter.

But for a user-facing API I'd choose a token bucket, and this is the part I care about most. Real users are bursty – opening an app fires six or eight calls at once, then nothing for five minutes. A sliding window punishes that even though the average is fine. A token bucket lets the burst through up to the bucket size while still capping the long-run rate. Two numbers control it: capacity is how much burst you tolerate, refill rate is the sustained limit.

I refill lazily rather than with a timer – compute the tokens that should have accrued since the last request. With a million users, a timer each would be the whole problem.

All four fit behind one `tryAcquire(key, now, cost)` method, and I pass `now` in rather than calling `Date.now()` inside, so the whole thing is testable without sleeping. Cost matters too – a search and a bulk export shouldn't both count as one.

One thing I'd add: the rejection should be a 429 with a `Retry-After`, not just a false. Without it clients retry immediately and you've replaced real load with rejection load."

16. Follow-ups – distributed rate limiting

Follow-up	జవాబు	మూరే classes
"Endpoint వారీగా వేరే limits"	<code>keyOf</code> ని <code>\${userId}:\${path}</code> చేయడం	0

Follow-up	జవాబు	మారే classes
"Premium users కి ఎక్కువ"	Tier ని బట్టి వేరే limiter, లేదా <code>limitFor(key)</code>	Strategy lo ఒక lookup
"Global limit కూడా కావాలి"	రెండే <code>RateLimiter</code> , key స్థిరంగా <code>'global'</code>	0 కొత్త concepts
"Burst ని పూర్తిగా ఆపాలి"	<code>capacity = refillPerSec</code> – bucket పెరగదు	0
"50 servers"	కింద చూడండి	–

50 servers అయితే

"అప్పుడు ఇది ఇక data structure problem కాదు – ఇది **distributed state** problem. మూడు ఎంపికలు:

1. Redis lo counter. అందరూ ఒకే చోట చూస్తారు, కాబట్టి ఖచ్చితం. ఖర్చు – ప్రతి request కి ఒక network hop, మరియు Redis ఒక single point of failure. Increment మరియు expiry ఒకే atomic ఆపరేషన్ కావాలి, లేకపోతే మళ్ళీ check-then-act race. `INCR` + `EXPIRE` ని ఒక Lua script lo పెట్టడం లేదా `INCR` తర్వాత మొదటిసారి `EXPIRE` పెట్టడం.

2. Local limits, విభజించి. 50 servers, హద్దు 1000 → ఒక్కొక్కరికి 20. Network hop లేదు, కానీ load balancing సమానంగా లేకపోతే కొందరు ముందే ఆగిపోతారు, మరికొందరికి quota మిగిలిపోతుంది.

3. మధ్యేమార్గం – local bucket + periodic sync. ప్రతి server తన సొంత లెక్క ఉంచుకుని, ప్రతి సెకనుకీ Redis తో సరిచూసుకుంటుంది. దాదాపు ఖచ్చితం, network hop ప్రతి request కీ కాదు. **నేను దీన్నే ఎంచుకుంటాను**, ఎందుకంటే rate limiting lo కొంచెం అస్పష్టత ఆమోదయోగ్యం, కానీ ప్రతి request కీ ఒక Redis hop కాదు."

Redis పడిపోతే? – "Fail open నా fail closed నా అనేది ఒక business నిర్ణయం. దుర్వినియోగం ఆపడం లక్ష్యం అయితే fail closed. అందుబాటు ముఖ్యం అయితే fail open, మరియు local bucket కి తిరిగి వెళ్ళడం. **నేను ఈ ప్రశ్నని interviewer ని అడుగుతాను** – ఇది నేను నిర్ణయించాల్సినది కాదు."

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
సరిహద్దులు దాడికి గురవుతాయి	Fixed window boundary burst (§4)	Pagination, cursor, batch జాబులు
ఖచ్చితత్వాన్ని memory కోసం వదులుకోవడం	Log → counter (§7)	Bloom filter, HyperLogLog, sampling
ఊహని నమ్మకుండా కొలవడం	అంచనా దిశని కొలవడం (§8)	ప్రతి performance వాదనలో

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
Lazy computation	Timer లేకుండా refill (§9)	TTL expiry, garbage collection, billing
సమయాన్ని inject చేయడం	<code>now</code> ఒక parameter (§11)	Cache TTL, scheduler, session
తిరస్కరణ ఒక API	429 + <code>Retry-After</code> (§12)	ఏ error response lo అయినా

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 LRU & LFU Cache · 03 Rate Limiter · 06 Elevator.**

తర్వాతివి: **04 BookMyShow** (seat locking = Parking Lot §7 యొక్క పెద్ద రూపం), **05 Splitwise.**

`clock` injection ఇప్పుడు మూడు docs lo వచ్చింది – Cache §15, ఇక్కడ §11, Elevator యొక్క `step()`.

అది యాదృచ్ఛికం కాదు; అది ఒక **సార్వత్రిక** నమూనా. వేగవంతమైన revision కోసం –

`LLD_Design_Problems_Telugu.pdf` lo Problem 08.

*Rate Limiter – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, memory కొలత, మరియు 5,000-
నమూనాల సరిపోలిక నిజంగా `node` lo run చేసి తీసినవే ✓*